

- ↳ How the parameters are estimated from data?
- ↳ λ in previous lectures.

Lecture 7.1: Conditional Random Fields

Inference Vs Learning

Image Denoising:

$$p(\underbrace{x_1, \dots, x_{100}}_{\text{Denoised image}}) = \frac{1}{Z} \exp \left\{ \sum_i \psi_i(x_i) + \sum_{i,j} \psi_{ij}(x_i, x_j) \right\}$$

Inference:

↳ Marginal distributions: $p(x_i) = \sum_{x/x_i} p(x_1, \dots, x_{100})$

↳ MAP solution: $x_1^*, \dots, x_{100}^* = \underset{x_1, \dots, x_{100}}{\operatorname{argmax}} p(x_1, \dots, x_{100})$

Note:

- ↳ In literature, potentials are sometimes defined as -ev log factors i.e low values of potentials give high probability. Thinking about them as Energy.
- ↳ Here, sign is omitted for simplicity i.e High values of potentials give high probability.

Learning:

↳ Estimate parameters (here λ) from the dataset

↳ If we use MRF then we can't define the learning problem. Why?

↳ Because we are looking at one model instantiation. In Image denoising, we are looking at one image that we want denoise & associate each pixel with a random variable.

↳ For learning we need to use a large dataset & not just a particular instance.

↳ That's why we need to formulate conditionally and this leads us to structured output learning.

$$f_w: X \rightarrow Y \rightarrow \text{Complex objects}$$

can be anything

Conditional Random Field:

↳ Make conditioning of output Y on input X & parameters w explicit (here $w = \lambda$)

$$p(y|x, w) = \frac{1}{Z} \exp \left\{ \sum_i \psi_i(\overset{\text{noising image}}{\underset{\text{Denoised image}}{x_i}}, \overset{\text{noising pixel}}{\underset{\text{clean image / grid truth}}{y_i}} \right\} + \lambda \sum_{i \sim j} \psi_i(x_i, y_i, x_j, y_j) \}$$

Estimate w from Dataset $\{(x^1, y^1) \dots (x^N, y^N)\}$

In general;

$$p(y|x, w) = \frac{1}{Z(x, w)} \exp \left\{ \langle \underset{\text{Concatenation of unary \& pairwise}}{w}, \underset{\text{Inner product}}{\psi_i(x, y)} \rangle \right\}$$

Note:

Where, x is input
 y is the output.
 In MRF x is the output & input is implicit in the definition of potentials.

feature function:

$$\psi(x, y): X \times R^M \rightarrow R^D \quad \text{No. of output vars. (Pixels in image)}$$

$$\Psi(x, y) = (\psi(x, y_1), \dots, \psi(x, y_k))$$

Parameter vector:

$$w \in R^D$$

Partition function:

$$Z(x, w) = \sum_y \exp \{ \langle w, \Psi(x, y) \rangle \}$$

Lecture 7.2: Parameter Estimation

Goal:

Maximize likelihood of y conditioned on inputs x wrt w , assuming independent & identically distributed data (IID)

$$\hat{w}_{ML} = \underset{w \in R^D}{\operatorname{argmax}} p(y|x, w)$$

$$p(y|x, w) = \prod_{n=1}^N p(y^n | x^n, w)$$

Find vector $\hat{w}_{ML}$ such that $p_{\text{model}}(y|x, \hat{w}_{ML}) \approx p_{\text{data}}(y|x)$
 we can formulate above as:

$$\hat{w}_{ML} = \underset{w \in R^D}{\operatorname{argmin}} \mathcal{L}(w)$$

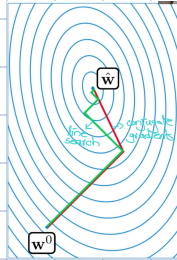
$$\mathcal{L}(w) = - \sum_{n=1}^N \log p(y^n | x^n, w)$$

$$\begin{aligned}
&= -\sum_{n=1}^N \left[\log \frac{1}{z(x^n, w)} \exp \left\{ \langle w, \psi_i(x^n, y^n) \rangle \right\} \right] \\
&= -\sum_{n=1}^N \left[-\log z(x^n, w) + \langle w, \psi(x^n, y^n) \rangle \right] \\
&= -\sum_{n=1}^N \left[\langle w, \psi(x^n, y^n) \rangle - \log \sum_{y \in Y} \exp \left\{ \langle w, \psi_i(x^n, y) \rangle \right\} \right]
\end{aligned}$$

swap & open z
↑

Do gradient descent.

We need $\nabla_w L(w)$;



$$\begin{aligned}
\nabla_w L(w) &= -\sum_{n=1}^N \left[\frac{\psi(x^n, y^n) - \sum_y \exp \left\{ \langle w, \psi_i(x^n, y) \rangle \right\} \psi_i(x^n, y)}{\sum_y \exp \left\{ \langle w, \psi_i(x^n, y) \rangle \right\}} \right] \\
&= -\sum_{n=1}^N \left[\psi(x^n, y^n) - \sum_y \frac{\exp \left\{ \langle w, \psi_i(x^n, y) \rangle \right\}}{\sum_{y'} \exp \left\{ \langle w, \psi_i(x^n, y') \rangle \right\}} \psi_i(x^n, y) \right] \\
&= -\sum_{n=1}^N \left[\psi(x^n, y^n) - \sum_y p(y | x^n, w) \psi_i(x^n, y) \right] \\
&= -\sum_{n=1}^N \left[\underbrace{\psi(x^n, y^n)}_{\text{Data features}} - \underbrace{\mathbb{E}_{y \sim p(y | x^n, w)} \psi_i(x^n, y)}_{\text{Expectation over predicted features}} \right]
\end{aligned}$$

→ pull inside of $\sum_y$

↳ Model is make true features & predicted as close as possible

↳ This convex but only as long as w is linear

When is $L(w)$ minimal?

$$E_{y \sim p(y|x^n, w)} \psi_i(x^n, y) = \psi(x^n, y^n) \Rightarrow \nabla_w L(w) = 0$$

Interpretation:

↳ We aim at expectation matching but discriminatively for $x \in \{x^1, \dots, x^n\}$, which is known data.

↳ Simply put, expectation of features under the model must be same to the features of observations

↳ $L(w)$ is convex (Hessian positive semi-definite). This is true only if $p(y|x, w)$ is log-linear in $w \in \mathbb{R}^D$

Computational Complexity:

$$L(w) = -\sum_{n=1}^N \left[\langle w, \psi(x^n, y^n) \rangle - \log \sum_{y \in Y} \exp \{ \langle w, \psi_i(x^n, y) \rangle \} \right]$$

$$\nabla_w L(w) = -\sum_{n=1}^N \left[\psi(x^n, y^n) - \sum_y p(y|x^n, w) \psi_i(x^n, y) \right]$$

Problem:

Y is exponentially large!

↳ Binary image segmentation: $|Y| = 2^{640 \times 480}$

The sum $\sum_y$ is over 10^{92475} . More than atoms in the universe

Concretely, $O(NCD)$

N : no. of samples (≈ 100 to 1000000) C : labels per output (2 to 100)
 M : no. of output nodes (≈ 100 to 1000000) D : dim of feature space

Lets reduce complexity.

↳ Starting with $\sum_y$:

We know in graphical models, features decompose as:

$$\psi(x, y) = [\psi_1(x, y_1), \dots, \psi_k(x, y_k)]$$

$\psi(x, y)$ is just a big vector that is concatenation of all potentials.

Think of this as global features

This can be broken down to each potential that produces a feature

↳ The Z simplifies to:

$$\sum_y \exp\{\langle w, \psi(x^n, y) \rangle\} = \sum_y \exp\left\{\sum_k \langle w_k, \psi_k(x^n, y_k) \rangle\right\}$$

↳ Here, Y is subset of all output space. In pairwise potential, this is two adjacent pixels

$$= \sum_{y \in Y} \prod_k \exp\{\langle w_k, \psi_k(x^n, y_k) \rangle\}$$

↳ This can be efficiently approximated using message passing

We use belief propagation to compute marginals for x_i (prev $x_1, \dots, x_{i-1}$) & sum them up

Similarly, feature expectation simplifies as:

$$\sum_y P(y | x^n, w) \psi_i(x^n, y) = \sum_{y \sim p} P(y | x^n, w) \sum_k \psi_k(x^n, y_k)$$

↳ Sum potentials separately

$$= \sum_k \sum_{y \sim p} P(y | x^n, w) \psi_k(x^n, y_k)$$

↳ Depends on only a subset of y .

$$= \sum_k \mathbb{E}_{y_k \sim p(y_k | x^n, w)} \psi_k(x^n, y_k)$$

↳ Adjust accordingly so that it depends only on k .

$$= \sum_k \sum_{y_k} p(y_k | x^n, w) \psi_k(x^n, y_k)$$

K : Number of potentials $\underbrace{\quad}_K$

C : No. of labels $\underbrace{\quad}_{C^F}$

F : Largest order (A subset of signal) $\approx (2-3)$

Thus, $O(NKC^F D)$

There's one left: N !

↳ Adds complexity linearly (Sum of over all samples)

↳ This makes learning over a large dataset an issue.

↳ Now, what?

↳ Simplify the model $\Rightarrow$ Bad results

↳ Train on a subset of data $\Rightarrow$ Ignores Information

↳ Parallelize $\Rightarrow$ Doesn't lessens computation

↳ Stochastic Gradient Descent

↳ In each gradient step:

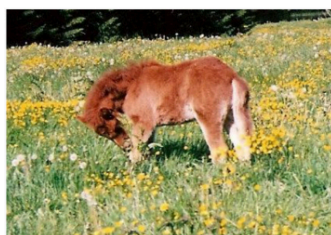
↳ Create random subset $D' \subset D$

$$\nabla w \approx - \sum_{(x^n, y^n) \in D'} [\psi(x^n, y^n) - \mathbb{E}_{y \sim p(y | x^n, w)} \psi(x^n, y)]$$

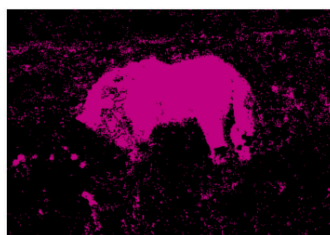
Applications

Semantic Segmentation:

- ▶ $\psi_i(\mathcal{X}, y_i) \in \mathbb{R}^{\approx 1000}$: local image features (e.g., bag of words, deep features)
→ $\langle \mathbf{w}_i, \psi_i(y_i) \rangle$: local classifier (ψ_i returns scaled 1-hot vector)
- ▶ $\psi_{ij}(y_i, y_j) = [y_i = y_j] \in \mathbb{R}^1$: test for same label
→ $\langle w_{ij}, \psi_{ij}(y_i, y_j) \rangle$: penalizer for label changes (if $w_{ij} > 0$)
- ▶ combined: $\arg\max_{\mathcal{Y}} p(\mathcal{Y} | \mathcal{X}, \mathbf{w})$ is smoothed version of local cues



original



local classification

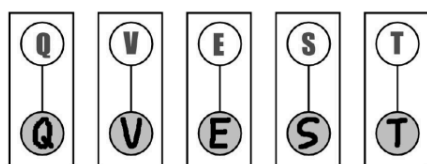


local + smoothness

Handwriting Recognition:

- ▶ $\psi_i(\mathcal{X}, y_i) \in \mathbb{R}^{\approx 1000}$: image representation (e.g., pixels, gradients)
→ $\langle \mathbf{w}_i, \psi_i(\mathcal{X}, y_i) \rangle$: local classifier for letters (ψ_i returns scaled 1-hot vector)
- ▶ $\psi_{ij}(y_i, y_j) = \mathbf{e}_{y_i} \mathbf{e}_{y_j}^T \in \mathbb{R}^{26 \times 26}$: letter/letter indicator (matrix with one element = 1)
→ $\langle \mathbf{w}_{ij}, \psi_{ij}(y_i, y_j) \rangle$: encourage/suppress letter combinations
- ▶ Combined: $\arg\max_{\mathcal{Y}} p(\mathcal{Y} | \mathcal{X}, \mathbf{w})$ is "corrected" version of local cues

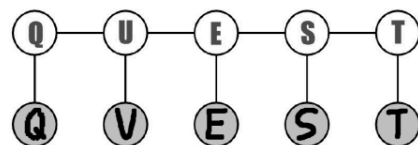
↳ which letter to occur likely after a letter?



local classification

Output

Input



local + "correction"

Pose Estimation:

- ▶ $\psi_i(\mathcal{X}, y_i) \in \mathbb{R}^{\approx 1000}$: image representation (e.g., HoG, deep features)
→ $\langle \mathbf{w}_i, \psi_i(\mathcal{X}, y_i) \rangle$: local confidence map (ψ_i returns scaled 1-hot vector)
- ▶ $\psi_{ij}(y_i, y_j) = \text{fit}(y_i, y_j) \in \mathbb{R}^1$: test for geometric fit / pose prior
→ $\langle w_{ij}, \psi_{ij}(y_i, y_j) \rangle$: penalizer for unrealistic poses
- ▶ Combined: $\arg\max_{\mathcal{Y}} p(\mathcal{Y} | \mathcal{X}, \mathbf{w})$ is sanitized version of local cues

↳ Head is more likely to be above body



original



local classification



local + geometry

Typical feature functions for CRFs in computer vision:

- ▶ **Unary terms** $\psi_i(\mathcal{X}, y_i)$: local representation, high-dimensional
→ $\langle \mathbf{w}_i, \psi_i(\mathcal{X}, y_i) \rangle$: local classifier (ψ_i returns scaled 1-hot vector which is non-zero only for the index y_i and $\dim(\mathbf{w}_i) = \text{number of labels of } y_i$)
- ▶ **Pairwise terms** $\psi_{ij}(y_i, y_j)$: prior knowledge, typically low-dimensional
→ $\langle w_{ij}, \psi_{ij}(y_i, y_j) \rangle$: penalize inconsistencies
- ▶ **Pairwise terms sometimes also depend on $\mathcal{X}$** : $\psi_{ij}(\mathcal{X}, y_i, y_j)$

Learning adjusts parameters:

- ▶ **Unary weights** $\mathbf{w}_i$: learn local linear classifiers
- ▶ **Pairwise weights** w_{ij} : learn importance of smoothing/penalization
- ▶ $\arg\max_{\mathcal{Y}} p(\mathcal{Y} | \mathcal{X}, \mathbf{w})$ is cleaned up version of local prediction

Sometimes, training the entire model at once is NOT easy.

↳ It is NOT so easy to specify features where parameters are linear in features. Example, feats need to be sophisticated to capture non-linear relations like x^2 or $\exp(x)$ etc.

↳ In order to tackle this, we add more features (increasing dimensionality)

Piece-Wise Training:

↳ Pre-train classifiers $p(y_i | x)$;
set $\psi_i(x, y_i) = \log p(y_i | x) \in \mathbb{R}$

↳ Learn 1-D weight per classifier: $\langle \mathbf{w}_i \psi(x, y_i) \rangle$

Pros:

↳ Lower dimensionality

↳ $\log p(y_i | x)$ can be stronger classifiers eg: CNNs

Cons:

If local classifiers are bad then CRF cannot fix this

Summary:

Given:

- ▶ Training set $\mathcal{D} = \{(\mathcal{X}^1, \mathcal{Y}^1), \dots, (\mathcal{X}^N, \mathcal{Y}^N)\}$ with $(\mathcal{X}^n, \mathcal{Y}^n) \stackrel{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathcal{X}, \mathcal{Y})$
- ▶ Feature function: $\psi(\mathcal{X}, \mathcal{Y}) : \mathbb{X} \times \mathbb{R}^M \rightarrow \mathbb{R}^D$

Task:

- ▶ Find parameter vector $\hat{\mathbf{w}}_{ML}$ such that

$$p_{\text{model}}(\mathcal{Y}|\mathcal{X}, \hat{\mathbf{w}}_{ML}) = \frac{1}{Z(\mathcal{X}, \hat{\mathbf{w}}_{ML})} \exp \{ \langle \hat{\mathbf{w}}_{ML}, \psi(\mathcal{X}, \mathcal{Y}) \rangle \} \approx p_{\text{data}}(\mathcal{Y}|\mathcal{X})$$

Minimize negative conditional log-likelihood:

$$\mathcal{L}(\mathbf{w}) = - \sum_{n=1}^N \left[\langle \mathbf{w}, \psi(\mathcal{X}^n, \mathcal{Y}^n) \rangle - \log \sum_{\mathcal{Y}} \exp \{ \langle \mathbf{w}, \psi(\mathcal{X}^n, \mathcal{Y}) \rangle \} \right]$$

- ▶ Convex optimization problem $\rightarrow$ gradient descent leads to global optimum
- ▶ Training needs repeated runs of probabilistic inference $\Rightarrow$ must be fast

Gradient of negative conditional log-likelihood:

$$\mathcal{L}(\mathbf{w}) = - \sum_{n=1}^N \left[\langle \mathbf{w}, \psi(\mathcal{X}^n, \mathcal{Y}^n) \rangle - \log \sum_{\mathcal{Y} \in \mathbb{Y}} \exp \{ \langle \mathbf{w}, \psi(\mathcal{X}^n, \mathcal{Y}) \rangle \} \right]$$

Problem	Solution	Method
$ \mathbb{Y} $ too large	exploit structure	belief propagation
N too large	mini-batches	stochastic gradient descent
D too large	trained ψ	piece-wise training

Lecture 7.3: Deep Structured Models

↳ So far models are log linear.

↳ The parameters appear log-linear fashion in the model equation.
↳ $\log(y) = \mathbf{w} \cdot \mathbf{x}$

↳ The model simply weighs each feature. By using a NN for as feature functions, we can model complex while still being log-linear.

↳ Update Params of nn & pmodel jointly.

Log-linear Models:

$$p(y|x, w) = \frac{1}{Z(x, w)} \exp \left\{ \underbrace{\langle w, \psi_i(x, y) \rangle}_{\text{Train them}} \right\}$$

↳ After taking log, we do dot product b/w w & features

↳ So, features must do the heavy lifting.

↳ Ideally, we want learn features themselves i.e. parametrized feature functions themselves, where these parameters do not appear in a linear function.

↳ We want to update not only relative weighting of these features as in log-linear models but also update these parameters of the features themselves.

↳ This is called "Deep structured models"

Deep structured models:

$$p(y|x, w) = \frac{1}{Z(x, w)} \exp \left\{ \underbrace{\psi_i(x, y, w)}_{\text{NN}} \right\}$$

Negative log likelihood & gradient

$$\nabla_w L(w) = - \sum_{n=1}^N \left[\nabla_w \psi(x^n, y^n, w) - \sum_y p(y|x^n, w) \nabla_w \psi_i(x^n, y, w) \right]$$

$$L(w) = - \sum_{n=1}^N \left[\psi(x^n, y^n, w) - \log \sum_{y \in Y} \exp \left\{ \psi_i(x^n, y, w) \right\} \right]$$

↳ Other terms calculated similar to before

↳ Similarly, sums decompose:

$$\psi(x, y, w) = \sum_k \psi_k(x, y_k, w)$$

Algorithm:

1. Forward pass to compute $\psi_k(x, y_k, w)$
2. Backward pass to compute gradients $\nabla_w \psi_k(x, y_k, w)$
3. Compute marginals using message passing
4. Update parameters w .

↳ This is very slow. So:

Unrolling Inference:

Idea:

- ↳ Consider inference as sequence of small computations
- ↳ "Unroll" a fixed number of inference iterations similar to RNN.
- ↳ Compute gradients using automatic differentiation

Remarks:

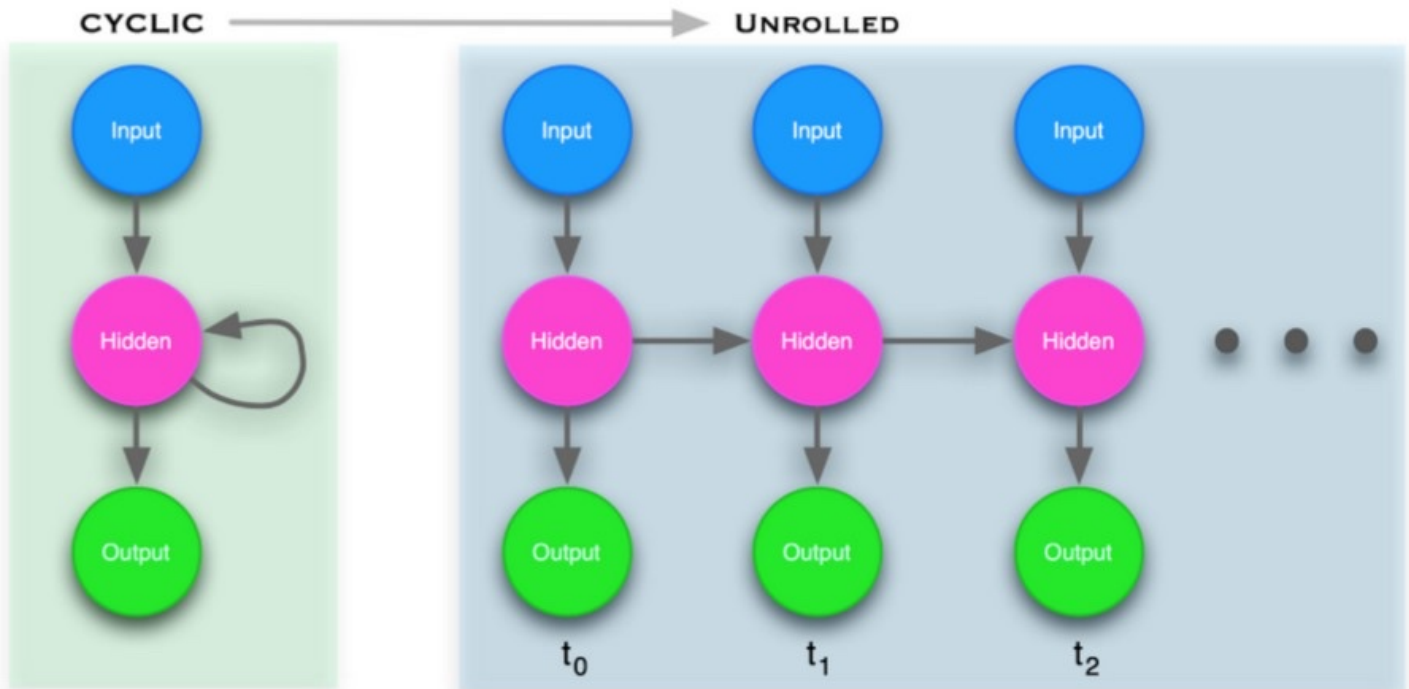
- ↳ Pused deterministic approach, giving up probabilistic view point
- ↳ fast enough for efficient training in deep models
- ↳ Effectively integrates structure of the problem into architecture of the network

Unrolled inference:

↳ Run belief propagation for fixed iterations

↳ Unroll them & treat each as a deep neural network layer

$$\text{Belief}(x_i^{(t)}) = \psi_i(x_i) + \sum_{j \in \mathcal{N}_i} \psi_{ij}(x_i, x_j) \times \text{Belief}(x_j^{(t-1)})$$



To compare with DL methods:

- **Initialize weights**

We take an input $\rightarrow$ perform computations $\rightarrow$ get the output $\rightarrow$ compute the loss.

This loss is backpropagated through the graph built during the "computation" phase.

Now, similar to this, we do the following for N times:

- **Initialize beliefs**

We take the input $\rightarrow$ perform computations $\rightarrow$ update beliefs $\rightarrow$

perform computations $\rightarrow$ update beliefs $\rightarrow$

perform computations $\rightarrow$ get the output $\rightarrow$ compute the loss.

Now, we "unroll" this process and backpropagate for SGD.

Each iteration is treated as a layer meaning it has its own computation graph.

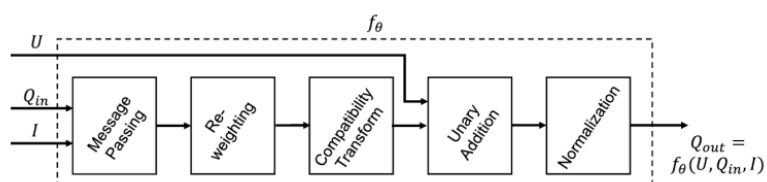
Our iterations are also fixed.

Examples:

Conditional Random as Recurrent Neural Networks

$$E(\mathbf{x}) = \sum_i \psi_u(x_i) + \sum_{i < j} \psi_p(x_i, x_j), \quad (1)$$

$$\psi_p(x_i, x_j) = \mu(x_i, x_j) \sum_{m=1}^M w^{(m)} k_G^{(m)}(\mathbf{f}_i, \mathbf{f}_j), \quad (2)$$



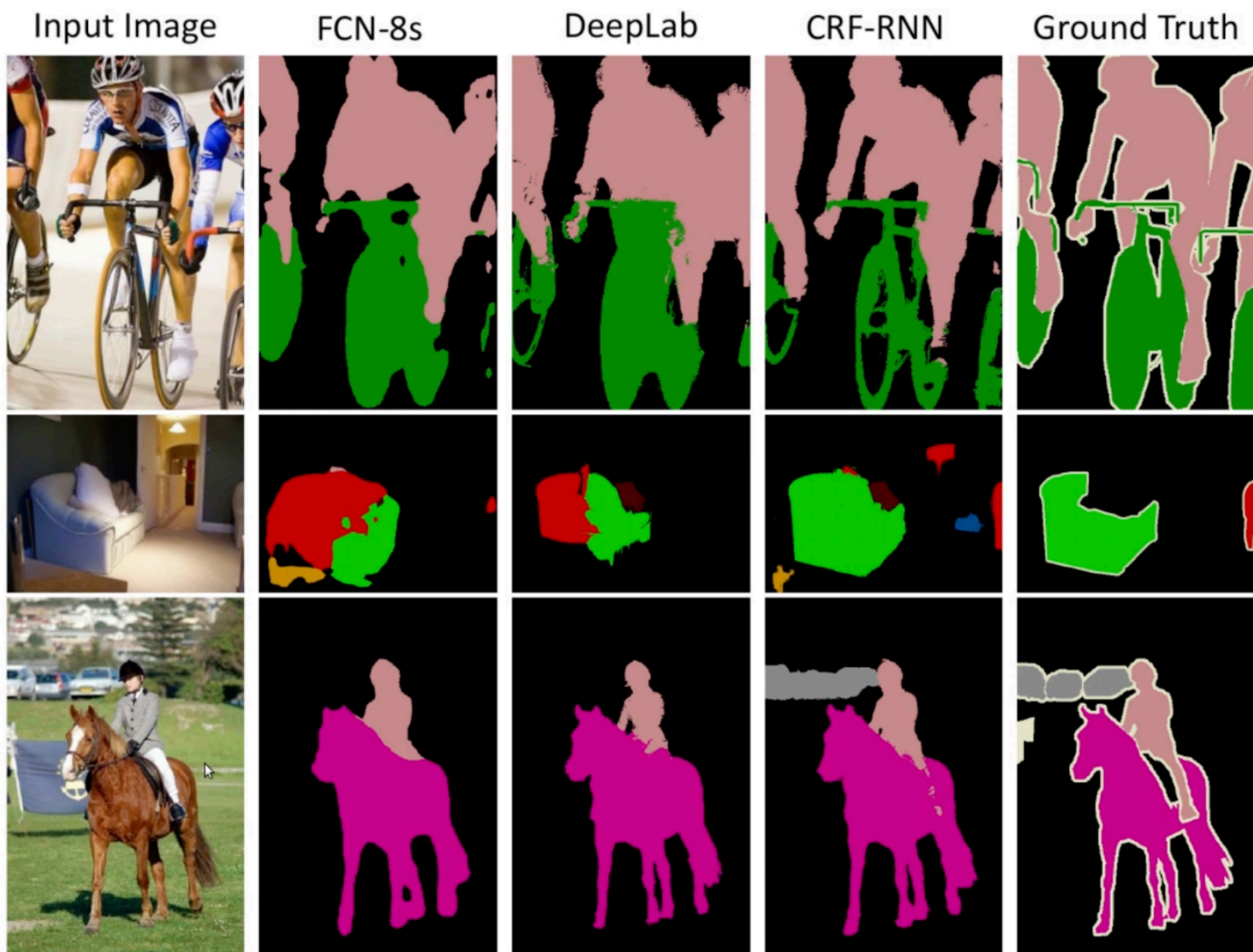
Algorithm 1 Mean-field in dense CRFs [29], broken down to common CNN operations.

```

 $Q_i(l) \leftarrow \frac{1}{Z_i} \exp(U_i(l))$  for all  $i$  ▷ Initialization
while not converged do
     $\tilde{Q}_i^{(m)}(l) \leftarrow \sum_{j \neq i} k^{(m)}(\mathbf{f}_i, \mathbf{f}_j) Q_j(l)$  for all  $m$  ▷ Message Passing
     $\tilde{Q}_i(l) \leftarrow \sum_m w^{(m)} \tilde{Q}_i^{(m)}(l)$  ▷ Weighting Filter Outputs
     $\hat{Q}_i(l) \leftarrow \sum_{l' \in \mathcal{L}} \mu(l, l') \tilde{Q}_i(l')$  ▷ Compatibility Transform
     $\check{Q}_i(l) \leftarrow U_i(l) - \hat{Q}_i(l)$  ▷ Adding Unary Potentials
     $Q_i \leftarrow \frac{1}{Z_i} \exp(\check{Q}_i(l))$  ▷ Normalizing

```

end while



RayNet: Learning Volumetric 3D Reconstruction

Distribution over voxel occupancies:

Corresponding factor graph:

$$p(\mathbf{o}) = \frac{1}{Z} \prod_{i \in \mathcal{X}} \underbrace{\varphi_i(o_i)}_{\text{unary}} \prod_{r \in \mathcal{R}} \underbrace{\psi_r(\mathbf{o}_r)}_{\text{ray}}$$

$$\varphi_i(o_i) = \gamma^{o_i} (1 - \gamma)^{1-o_i}$$

$$\psi_r(\mathbf{o}_r) = \sum_{i=1}^{N_r} o_i^r \prod_{j < i} (1 - o_j^r) s_i^r$$

